

Estilos de integración de APIs: guía comparativa de SOAP, REST, gRPC y GraphQL para decidir con criterio

Steven Ayala
Universidad Comunera
Asunción, Paraguay
stevengracia@s.ucom.edu.py

Ana Duarte
Universidad Comunera
Asunción, Paraguay
anaduarte@s.ucom.edu.py

Resumen—Cuando dos sistemas necesitan intercambiar información, alguien debe decidir cómo se hablarán. Esa decisión —el estilo de integración de la API— condiciona el costo de desarrollo, el rendimiento percibido por el usuario y la facilidad con que la solución podrá evolucionar durante años. Este artículo compara los cuatro estilos dominantes en la industria: SOAP, REST, gRPC y GraphQL. Mediante una revisión narrativa de especificaciones oficiales, literatura técnica de referencia y guías de diseño de sistemas ampliamente utilizadas, se construye una rejilla de nueve criterios de evaluación y se aplica a cada estilo. Los resultados muestran que ningún estilo domina a los demás en todas las dimensiones: SOAP aporta contratos rígidos y garantías empresariales a costa de verbosidad; REST ofrece simplicidad y aprovechamiento de la infraestructura web a costa de precisión en los datos que devuelve; gRPC entrega latencia baja y contratos estrictos a costa de interoperabilidad con navegadores; y GraphQL resuelve el problema del cliente que necesita datos a la medida a costa de complejidad operativa. Se concluye con criterios prácticos de elección y con la evidencia de que las arquitecturas reales combinan varios estilos en lugar de escoger uno solo.

Palabras clave—estilos de API, integración de sistemas, REST, SOAP, GraphQL, gRPC, arquitectura de software

I. INTRODUCCIÓN

Toda organización que crece termina enfrentando el mismo problema: los sistemas que compró, los que construyó y los que heredó deben intercambiar información entre sí. El sistema de facturación necesita datos del catálogo de productos, la aplicación móvil necesita el saldo del cliente y el proveedor externo necesita confirmar un envío. La pieza que hace posible ese intercambio es la interfaz de programación de aplicaciones o API, y la forma en que se diseña esa interfaz se conoce como estilo de integración.

Conviene una analogía sencilla para quienes se acercan al tema desde el lado del negocio. Una API es un contrato entre dos partes: define qué se puede pedir, cómo se debe pedir y qué se recibirá a cambio. Igual que ocurre con los contratos legales, existen distintos formatos. Algunos son extensos, detallados y obligan a especificar cada cláusula por adelantado; otros son breves, flexibles y confían en convenciones compartidas. Ninguno es intrínsecamente mejor: un contrato de arrendamiento de dos páginas es adecuado para alquilar una bodega, pero sería temerario para financiar una obra pública. El estilo de integración funciona igual.

La elección importa más de lo que sugiere su apariencia técnica. Un estilo mal escogido no falla el primer día, sino al año siguiente, cuando el equipo descubre que la aplicación móvil hace catorce llamadas para pintar una pantalla, que cada cambio en el servidor obliga a desplegar seis consumidores,

o que la factura mensual de la nube creció porque la infraestructura de caché quedó inutilizable. Amundsen (2023) plantea precisamente que los estilos de API no son alternativas de moda sino decisiones arquitectónicas con consecuencias duraderas, y que el error frecuente consiste en adoptar el estilo que domina la conversación pública en lugar del que corresponde al problema.

Este artículo responde a las cuatro preguntas que suelen surgir en cualquier discusión sobre el tema. ¿Qué estilos existen realmente disponibles hoy? ¿Cuáles son las características que distinguen a cada uno? ¿Qué limitaciones traen consigo, incluidas las que la documentación oficial no destaca? ¿Y qué criterios permiten escoger con fundamento? El alcance se limita a los cuatro estilos con mayor adopción documentada: SOAP, REST, gRPC y GraphQL. Se excluyen deliberadamente los estilos orientados a eventos, como la mensajería asíncrona con colas o la publicación y suscripción, porque resuelven un problema distinto —la comunicación desacoplada en el tiempo— y merecen un tratamiento propio.

La audiencia esperada es mixta. Por eso el texto evita el detalle de implementación y privilegia el razonamiento: qué gana y qué pierde cada quien al escoger un camino. Quien busque ejemplos de código encontrará mejores fuentes en las referencias; quien busque decidir encontrará aquí una rejilla de criterios aplicable a un proyecto personal, a una iniciativa departamental o a una estrategia corporativa.

II. MÉTODOS

Este trabajo es una revisión narrativa de literatura. No se ejecutaron mediciones propias de rendimiento ni se implementaron prototipos comparativos; el objetivo es sintetizar y ordenar conocimiento ya publicado para hacerlo accesible a una audiencia amplia.

Las fuentes se organizaron en tres niveles según su autoridad. En el primer nivel se consultaron las especificaciones oficiales, por ser la única descripción normativa de cada tecnología: la especificación SOAP 1.2 del W3C (2007), la semántica de HTTP formalizada en el RFC 9110 (Fielding et al., 2022), la especificación de GraphQL publicada por la GraphQL Foundation (2021) y la documentación de gRPC y Protocol Buffers mantenida por sus autores (gRPC Authors, 2024; Google, 2024). En el segundo nivel se incorporó literatura técnica revisada y ampliamente citada: la tesis doctoral de Fielding (2000), que introdujo REST como estilo arquitectónico; el catálogo de patrones de Digneau (2011) para servicios SOAP/WSDL y REST; el trabajo de Richardson y Amundsen (2013) sobre diseño de APIs web; el análisis formal de Hartig y Pérez (2018) sobre la complejidad computacional de las consultas GraphQL; y la obra de Newman (2021) sobre integración entre microservicios. En el tercer nivel se usaron guías

profesionales de amplia circulación, señaladas en la bibliografía del curso, como el comparativo de AltexSoft (2024), las guías de ByteByteGo (2024) y el System Design Handbook (2024); estas se emplearon para identificar prácticas de la industria y contrastar la experiencia reportada por equipos reales, no como evidencia normativa.

El criterio de inclusión fue doble: que la fuente describiera el estilo con suficiente precisión técnica y que estuviera disponible públicamente para que la persona lectora pueda verificarla. Se descartaron entradas de blog sin autoría identificable y comparativas promocionales de proveedores.

A partir de esas fuentes se construyó una rejilla de nueve criterios de evaluación, escogidos porque aparecen de forma recurrente en la literatura y porque cada uno tiene consecuencias observables en un proyecto: (1) modelo de interacción, es decir, si el cliente manipula recursos, invoca procedimientos o declara los datos que necesita; (2) contrato y tipado, o cuán formal y verificable es el acuerdo entre las partes; (3) transporte y formato del mensaje; (4) evolución y versionado, esto es, qué ocurre cuando el servicio cambia; (5) rendimiento y tamaño de la carga útil; (6) capacidad de aprovechar caché; (7) modelo de seguridad; (8) herramientas disponibles y curva de aprendizaje; y (9) madurez del ecosistema y disponibilidad de talento.

Cada estilo se describió de manera individual y luego se evaluó contra la rejilla. Los resultados se consolidaron en tres tablas comparativas y se discutieron a la luz de escenarios de uso concretos.

III. RESULTADOS

A. SOAP: el contrato notariado

SOAP, cuyo nombre proviene de Simple Object Access Protocol, es un protocolo de intercambio de mensajes estructurados definido por el W3C (2007). Su unidad de trabajo es un sobre XML con una cabecera y un cuerpo, transportado normalmente sobre HTTP, aunque la especificación permite otros transportes. La operación disponible, sus parámetros y sus posibles errores se declaran en un documento WSDL, que actúa como contrato formal y legible por máquina.

La característica que define a SOAP es la rigidez deliberada. El WSDL permite generar automáticamente el código del cliente y del servidor, de modo que un incumplimiento del contrato se detecta al compilar y no en producción. Sobre esa base se construyó la familia de especificaciones WS-*, que añade capacidades que otros estilos delegan en la infraestructura: firma y cifrado a nivel de mensaje con WS-Security, entrega confiable con WS-ReliableMessaging y transacciones distribuidas con WS-AtomicTransaction. Daigneau (2011) documenta cómo estos mecanismos permitieron a la banca y a los seguros construir integraciones auditables mucho antes de que existieran alternativas equivalentes en el mundo REST.

Las limitaciones son igual de nítidas. El sobre XML es verboso: una consulta de saldo puede ocupar varios kilobytes de los cuales apenas una fracción es información útil. El procesamiento requiere análisis sintáctico de XML, más costoso que el de formatos binarios o JSON. La curva de aprendizaje es pronunciada y el soporte en lenguajes modernos y en navegadores es marginal. Y la rigidez que protege también inmoviliza: modificar el contrato obliga a coordinar a todos los consumidores. Hoy SOAP sobrevive

sobre todo como tecnología heredada en sectores regulados, y esa condición no es un defecto sino el reflejo de que cumple bien un requisito específico.

B. REST: las convenciones de la web

REST no es un protocolo sino un estilo arquitectónico descrito por Fielding (2000) a partir de las restricciones que hicieron escalable a la web: interacción cliente-servidor, ausencia de estado en el servidor entre peticiones, capacidad de caché, interfaz uniforme y sistema en capas. En la práctica cotidiana, una API REST expone recursos identificados por URL y los manipula con los métodos de HTTP definidos en el RFC 9110: GET para leer, POST para crear, PUT o PATCH para modificar y DELETE para eliminar. El formato habitual de intercambio es JSON.

Su gran virtud es que no inventa nada. Al apoyarse en HTTP, hereda gratuitamente toda la infraestructura de internet: caché en navegadores y en redes de distribución de contenido, códigos de estado comprensibles, balanceadores, proxies inversos, cortafuegos y herramientas de diagnóstico que cualquier persona con experiencia web ya conoce. Esa familiaridad reduce el costo de adopción de forma drástica, lo que explica que sea el estilo predominante para APIs públicas (AltexSoft, 2024).

Sus limitaciones surgen del mismo origen. La primera es la imprecisión de los datos devueltos: un recurso tiene una representación fija, de modo que el cliente recibe campos que no necesita —sobrecarga o *over-fetching*— o debe hacer varias llamadas para completar lo que sí necesita —insuficiencia o *under-fetching*—. Una pantalla de perfil que muestre el usuario, sus últimos pedidos y sus direcciones puede requerir tres peticiones encadenadas. La segunda es que REST carece de un contrato obligatorio; OpenAPI cubre ese vacío, pero es una convención añadida y no una garantía del estilo. La tercera es que la disciplina real es escasa: Fowler (2010), al popularizar el modelo de madurez de Richardson, mostró que muchas APIs autodenominadas REST se detienen en el uso de URLs y verbos y nunca alcanzan la interfaz uniforme completa.

C. gRPC: llamadas remotas de alto rendimiento

gRPC materializa el estilo de llamada a procedimiento remoto o RPC: el cliente invoca lo que parece una función local y el marco de trabajo se encarga de la comunicación. El contrato se escribe en un archivo *.proto* con Protocol Buffers, del cual se generan cliente y servidor en más de una decena de lenguajes (Google, 2024). El transporte es HTTP/2 y los mensajes viajan en formato binario.

Las tres decisiones anteriores explican su rendimiento. La serialización binaria produce cargas útiles considerablemente menores que las de JSON o XML equivalentes. HTTP/2 multiplexa varias llamadas sobre una misma conexión y evita el bloqueo de cabecera de línea. Y el código generado elimina la escritura manual de clientes. A eso se añade el soporte nativo de cuatro modos de comunicación, incluidos el flujo continuo desde el servidor, desde el cliente y bidireccional, algo que REST solo consigue con mecanismos adicionales (gRPC Authors, 2024). Por eso Newman (2021) lo señala como opción natural para comunicación interna entre microservicios, donde la latencia acumulada de decenas de llamadas por petición sí se percibe.

Las limitaciones son de contexto más que de diseño. Los navegadores no pueden consumir gRPC directamente y

requieren una capa intermedia como gRPC-Web, lo que lo descarta como API pública para clientes web. El formato binario no es legible por humanos, de modo que depurar exige herramientas específicas en lugar de un simple *curl*. El acoplamiento al contrato es fuerte: aunque Protocol Buffers admite evolución compatible si se respetan reglas estrictas sobre los números de campo, romperlas provoca fallos difíciles de rastrear. Y la caché intermedia de HTTP no aplica, por lo que ese ahorro debe implementarse a mano.

D. GraphQL: el cliente pide exactamente lo que necesita

GraphQL invierte la responsabilidad. En lugar de que el servidor decida qué representación devuelve, el cliente envía una consulta declarativa que describe con precisión los campos requeridos y recibe una respuesta con esa forma exacta (GraphQL Foundation, 2021). El servicio expone un esquema tipado único, normalmente en un solo punto de entrada, y ese esquema es a la vez contrato, documentación y base para el autocompletado en las herramientas de desarrollo.

El problema que resuelve es concreto y frecuente: aplicaciones móviles y de página única que necesitan combinar datos de varias entidades en una sola pantalla y que, con REST, sufrirían sobrecarga o insuficiencia de datos. Una consulta única sustituye la cadena de peticiones y, sobre redes

móviles lentas, esa diferencia es perceptible. Además, el esquema permite agregar campos sin romper a los consumidores existentes, lo que en la práctica reduce la necesidad de versionar la API.

Sus limitaciones son principalmente operativas. Como toda la comunicación ocurre por POST hacia un mismo punto de entrada, la caché de HTTP deja de funcionar y debe reemplazarse por caché en el cliente o por capas especializadas. La observabilidad se complica: los códigos de estado y las métricas por ruta pierden significado cuando todas las peticiones comparten la misma URL, y un error puede llegar con estado 200. El riesgo más serio es de disponibilidad: Hartig y Pérez (2018) demostraron que una consulta anidada puede crecer de forma exponencial y que evaluarla es computacionalmente costoso, por lo que un servicio expuesto sin límites de profundidad, de complejidad ni presupuesto de consulta queda vulnerable a agotamiento de recursos. OWASP (2023) recoge este patrón entre los riesgos habituales de las APIs modernas. Finalmente, el problema conocido como N+1 obliga a introducir agrupadores de carga para evitar que una consulta elegante genere cientos de accesos a la base de datos.

E. Síntesis comparativa

TABLE I. TABLA I. COMPARACIÓN DE LOS CUATRO ESTILOS SEGÚN LA REJILLA DE NUEVE CRITERIOS

Criterio	SOAP	REST	gRPC	GraphQL
Modelo de interacción	Operaciones sobre mensajes	Recursos con verbos HTTP	Procedimientos remotos	Consulta declarativa
Contrato y tipado	WSDL obligatorio, fuerte	Opcional (OpenAPI), débil	.proto obligatorio, fuerte	Esquema obligatorio, fuerte
Transporte y formato	HTTP u otros; XML	HTTP/1.1 o HTTP/2; JSON	HTTP/2; binario (Protobuf)	HTTP; JSON
Evolución y versionado	Costosa: cambio coordinado	Por URL o cabecera	Compatible si se respetan reglas	Aditiva, con campos obsoletos
Rendimiento y carga útil	Bajo; mensajes grandes	Medio; JSON legible	Alto; mensajes compactos	Medio; evita datos sobrantes
Caché	No aprovecha caché HTTP	Nativa en toda la web	No aplica; manual	No aprovecha caché HTTP
Seguridad	WS-Security a nivel de mensaje	TLS, OAuth 2.0, claves de API	TLS, mTLS, tokens	TLS y OAuth 2.0 más límites de consulta
Herramientas y aprendizaje	Curva alta, herramientas antiguas	Curva baja, herramientas ubicuas	Curva media, generación de código	Curva media-alta, buen tooling
Madurez del ecosistema	Muy madura, en declive	Dominante y estable	Creciente, sólida en la nube	Madura en el frontend

TABLE II. TABLA II. LIMITACIONES Y COSTOS OCULTOS POR ESTILO

Estilo	Limitación principal	Costo oculto frecuente
SOAP	Verbosidad y complejidad del XML	Talento escaso; imposible de retirar en sistemas heredados
REST	Sobrecarga e insuficiencia de datos	Proliferación de rutas y llamadas encadenadas en móviles
gRPC	Inaccesible desde el navegador	Depuración difícil; caché y observabilidad hechas a mano
GraphQL	Complejidad operativa del servidor	Límites de consulta, agrupadores de carga y control de acceso por campo

TABLE III. TABLA III. MATRIZ DE DECISIÓN POR ESCENARIO

Escenario	Estilo recomendado	Razón principal
API pública para terceros	REST	Barrera de entrada mínima y caché aprovechable
Microservicios internos con latencia crítica	gRPC	Carga útil compacta y multiplexación sobre HTTP/2
Aplicación móvil con pantallas compuestas	GraphQL	Una consulta sustituye varias llamadas encadenadas
Integración bancaria o de seguros regulada	SOAP	Contrato formal, firma por mensaje y transaccionalidad
Telemetría o flujo continuo de datos	gRPC	Modos de comunicación en flujo nativos
Equipo pequeño sin plataforma dedicada	REST	Menor costo operativo y de aprendizaje

IV. DISCUSIÓN

A. Los criterios que realmente deciden

La comparación anterior sugiere una conclusión incómoda para quien busque una recomendación única: no existe un estilo superior. Existe correspondencia o falta de correspondencia entre un estilo y un contexto. Seis factores concentran la mayor parte del peso de la decisión.

El primero es quién consume la API. Si el consumidor es desconocido, externo y numeroso, la prioridad es la facilidad de adopción y REST gana casi por definición. Si el consumidor es otro equipo de la misma organización, se puede exigir un contrato estricto y aprovechar gRPC.

El segundo es cuánto acoplamiento resulta tolerable. Los contratos fuertes de SOAP y gRPC detectan errores temprano, pero obligan a coordinar despliegues; los contratos débiles de REST permiten avanzar por separado a costa de descubrir incompatibilidades más tarde.

El tercero es el perfil de latencia y volumen. Cuando una petición de usuario desencadena decenas de llamadas internas, unos pocos milisegundos por llamada se convierten en cientos, y ahí la eficiencia de serialización deja de ser un detalle.

El cuarto es la capacidad real del equipo. Adoptar GraphQL implica asumir de forma permanente el control de complejidad de consultas, la agrupación de cargas y la autorización campo por campo. Un equipo de tres personas rara vez puede sostener ese costo, y el System Design Handbook (2024) insiste en que la simplicidad operativa es una restricción tan legítima como el rendimiento.

El quinto es la gobernanza y el marco regulatorio. En sectores auditados, la posibilidad de firmar y cifrar el mensaje completo con independencia del canal —y de demostrarlo ante un auditor— puede pesar más que cualquier consideración de eficiencia.

El sexto es el costo total de operación. Perder la caché de HTTP significa construirla en otro lugar; perder la legibilidad de los mensajes significa invertir en herramientas de diagnóstico. Estos costos no aparecen en las comparativas de rendimiento, pero sí en el presupuesto del año siguiente.

B. La respuesta real es la combinación

Presentar los estilos como opciones excluyentes es un artificio didáctico. Las arquitecturas en producción los combinan según la frontera que atraviesa cada llamada, y esa es probablemente la observación más útil de este trabajo.

El patrón más extendido separa el interior del exterior: gRPC para la comunicación entre servicios internos, donde importan la latencia y el contrato estricto, y REST o GraphQL en el borde, donde importa que cualquier cliente pueda conectarse. Sobre esa base, el patrón *backend for frontend* añade una capa por tipo de cliente que traduce y agrega llamadas internas; cuando esa capa se implementa con GraphQL, el equipo móvil obtiene consultas a la medida sin que cada servicio interno deba exponer un esquema. Las organizaciones con sistemas heredados suelen añadir además una pasarela que traduce SOAP a REST, lo que permite modernizar los consumidores sin tocar un núcleo que nadie quiere arriesgar. La decisión, entonces, no es qué estilo adoptar para todo, sino qué estilo corresponde a cada frontera.

C. Errores frecuentes

De la literatura consultada emergen tres equivocaciones recurrentes que conviene nombrar.

La primera es llamar REST a lo que no lo es. Numerosas APIs exponen rutas como */obtenerUsuario* o */procesarPago* y usan POST para todo: son RPC sobre HTTP con nombres de REST, y al no respetar la interfaz uniforme pierden precisamente los beneficios que motivaron la elección (Fowler, 2010; Richardson y Amundsen, 2013).

La segunda es adoptar GraphQL por su elegancia en las demostraciones y desplegarlo sin límites de profundidad ni presupuesto de complejidad. La consecuencia previsible es un servicio que una sola consulta bien construida puede dejar fuera de servicio (Hartig y Pérez, 2018).

La tercera es intentar reemplazar SOAP por decreto. Las integraciones SOAP suelen sostener procesos críticos con consumidores que nadie inventarió; la estrategia sensata es envolverlas tras una fachada moderna y retirarlas por partes, no sustituirlas de una vez.

Un cuarto punto atraviesa a los cuatro estilos: la seguridad no la determina el estilo. TLS es obligatorio en todos los casos; la autorización delegada mediante OAuth 2.0 (Hardt, 2012) es aplicable a cualquiera de ellos; y los riesgos catalogados por OWASP (2023) —autorización rota a nivel de objeto, exposición excesiva de datos, consumo sin límites de recursos— aparecen con independencia de la tecnología escogida. Cambiar de estilo nunca resuelve un problema de diseño de autorización.

D. Limitaciones de este estudio

Tres advertencias son necesarias. Se trata de una revisión narrativa y no sistemática, por lo que la selección de fuentes, aunque explícita, no está exenta de sesgo. No se realizaron mediciones propias: las afirmaciones sobre rendimiento reproducen lo reportado por las fuentes y su magnitud depende siempre de la carga, el hardware y la implementación concreta. Y el panorama tecnológico se mueve: propuestas como gRPC-Web, la federación de esquemas GraphQL o el uso creciente de HTTP/3 pueden alterar parte de las conclusiones en pocos años.

V. CONCLUSIONES

Los cuatro estilos analizados responden a problemas distintos y ninguno domina a los demás en todas las dimensiones evaluadas. SOAP ofrece un contrato formal y garantías empresariales al precio de la verbosidad y la rigidez; REST ofrece adopción inmediata y aprovechamiento de la infraestructura web al precio de la imprecisión en los datos devueltos; gRPC ofrece rendimiento y contratos estrictos al precio de la interoperabilidad con el navegador; y GraphQL ofrece flexibilidad para el cliente al precio de una carga operativa considerable en el servidor.

Para quien deba decidir, la recomendación práctica se resume en tres movimientos. Primero, caracterizar el contexto antes que la tecnología: identificar quién consume, con qué latencia, bajo qué regulación y con qué equipo. Segundo, aplicar una rejilla explícita de criterios como la propuesta aquí, de modo que la decisión quede documentada y pueda revisarse cuando el contexto cambie. Tercero, aceptar que la respuesta correcta suele ser una combinación: un estilo para el interior del sistema, otro para el borde y una fachada para lo heredado.

La conclusión de fondo es que la pregunta no es cuál estilo es mejor, sino qué está optimizando la organización y qué está dispuesta a pagar por ello. Formulada así, la decisión deja de ser una discusión sobre tecnologías y se convierte en lo que siempre fue: una decisión de arquitectura.

REFERENCIAS

- Amundsen, M. (2023). Learning API styles. O'Reilly Media.
- AltexSoft. (2024). Comparing API architectural styles: SOAP vs REST vs GraphQL vs RPC. <https://www.altexsoft.com/blog/soap-vs-rest-vs-graphql-vs-rpc/>
- ByteByteGo. (2024). API and web development guides. <https://bytebytego.com/guides/api-web-development/>
- Daigneau, R. (2011). Service design patterns: Fundamental design solutions for SOAP/WSDL and RESTful web services. Addison-Wesley.
- Fielding, R. T. (2000). Architectural styles and the design of network-based software architectures [Tesis doctoral, University of California, Irvine].
- Fielding, R. T., Nottingham, M., y Reschke, J. (2022). HTTP semantics (RFC 9110). Internet Engineering Task Force. <https://doi.org/10.17487/RFC9110>
- Fowler, M. (2010, 18 de marzo). Richardson maturity model. <https://martinfowler.com/articles/richardsonMaturityModel.html>
- Google. (2024). Protocol Buffers documentation. <https://protobuf.dev/>
- GraphQL Foundation. (2021). GraphQL specification (October 2021 edition). <https://spec.graphql.org/October2021/>
- gRPC Authors. (2024). gRPC documentation. <https://grpc.io/docs/>
- Hardt, D. (2012). The OAuth 2.0 authorization framework (RFC 6749). Internet Engineering Task Force. <https://doi.org/10.17487/RFC6749>
- Hartig, O., y Pérez, J. (2018). Semantics and complexity of GraphQL. Proceedings of the 2018 World Wide Web Conference, 1155-1164. <https://doi.org/10.1145/3178876.3186014>
- Newman, S. (2021). Building microservices: Designing fine-grained systems (2.a ed.). O'Reilly Media.
- OWASP. (2023). OWASP API Security Top 10 - 2023. <https://owasp.org/API-Security/editions/2023/en/0x00-header/>
- Richardson, L., y Amundsen, M. (2013). RESTful Web APIs. O'Reilly Media.
- System Design Handbook. (2024). System design guides. <https://www.systemdesignhandbook.com/guides/system-design/>
- W3C. (2007). SOAP version 1.2 part 1: Messaging framework (2.a ed.). World Wide Web Consortium. <https://www.w3.org/TR/soap12-part1/>